

Hardware-aware Quantization/Mapping Strategies for Compute-in-Memory Accelerators

SHANSHI HUANG, HONGWU JIANG, and SHIMENG YU, Georgia Institute of Technology

The **emerging non-volatile memory (eNVM)** based mixed-signal **Compute-in-Memory (CIM)** accelerators are of great interest in today's AI accelerators design due to their high energy efficiency. Various CIM architectures and circuit-level designs have been proposed, showing superior hardware performance for **deep neural network (DNN)** acceleration. However, hardware-aware quantization strategies for CIM-based accelerators are not systematically explored. Since there are a variety of design options for neural network mapping on CIM systems while improper strategies may narrow the circuit-level design space and further limit the hardware performance, it is important to make a comprehensive early-stage design space exploration and find appropriate quantization/mapping strategies to achieve better hardware performance. In this paper, we provide a joint algorithm-hardware analysis and compare the system-level hardware performance for various design options, including quantization algorithms, data representation methods and **analog-to-digital converter (ADC)** configurations. This work aims to propose guidelines for choosing more hardware-friendly design options for chip architects. According to our evaluation results for CIFAR-10/100 and ImageNet classification, the properly chosen quantization approach and optimal mapping strategy (dynamic fixed-point quantization + 2's complement representation/shifted unsigned INT representation + optimized precision ADC) could achieve $\sim 2\times$ energy efficiency and $1.2\sim 1.6\times$ throughput with $5\%\sim 25\%$ reduced area overhead, compared to naïve strategy (fixed-point quantization + differential pair number representation + full precision ADC).

CCS Concepts: • **Hardware** → **Analysis and design of emerging devices and systems** • **Computing methodologies** → **Machine learning**;

Additional Key Words and Phrases: Compute-in-memory, deep neural network, design space exploration

ACM Reference format:

Shanshi Huang, Hongwu Jiang, and Shimeng Yu. 2023. Hardware-aware Quantization/Mapping Strategies for Compute-in-Memory Accelerators. *ACM Trans. Des. Autom. Electron. Syst.* 28, 3, Article 34 (March 2023), 23 pages.

<https://doi.org/10.1145/3569940>

1 INTRODUCTION

In recent years, deep learning algorithms (e.g., convolutional neural networks) have shown great success in many fields. However, as the scale of **deep neural network (DNN)** models grows, the number of parameters and computations increases dramatically. Consequently, traditional

This work is supported by ASCENT, one of the SRC/DARPA JUMP Centers.

Authors' address: S. Huang, H. Jiang, and S. Yu (corresponding author), School of Electrical and Computer Engineering, Georgia Institute of Technology, 791 Atlantic Dr NW, Atlanta, GA 30332, US; emails: {shuang406, hjiang318}@gatech.edu, shimeng.yu@ece.gatech.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2023 Association for Computing Machinery.

1084-4309/2023/03-ART34 \$15.00

<https://doi.org/10.1145/3569940>

computing architecture's throughput and energy efficiency are constrained by the massive data movement between the processing unit and the storage unit. This memory wall bottleneck necessitates alternative computer architectures to obtain better hardware efficiency. Therefore, there are many research efforts on developing memory-centric architectures. By integrating the computations into memory arrays, mixed-signal compute-in-memory architectures have shown impressive abilities in boosting the throughput and energy efficiency of deep learning algorithms [1]. The *in-situ* **vector-matrix multiplication (VMM)** in CIM crossbar arrays leads to lower computing energy, and the weight-stationary dataflow requires much less data transfer. Meanwhile, as emerging memories (such as **resistive random access memory (RRAM)** [2] and **ferroelectric field-effect transistor (FeFET)** [3]) offer high integration density and low operating energy costs, they are becoming promising candidates for CIM implementation. Besides, these emerging memories also benefit from the non-volatility feature, which could preserve data with no power supply. Thus, various CIM designs based on **emerging non-volatile memory (eNVM)** have been proposed for edge inference [4, 5, 6].

Even though there is a demanding interest for developing CIM accelerators, studies mostly focused on specific hardware design, which can be sorted into two categories: (1) circuit/device-level implementations (e.g., advanced memory technology and lightweight ADC design) to increase throughput and reduce energy consumption; (2) architecture-level optimizations (e.g., sparsity-aware computing and dataflow pipeline) to improve system efficiency. However, due to the crossbar structure of CIM array and its essentially analog processing manner, a scratch floating-point DNN model from a digit system cannot be directly mapped in CIM crossbar arrays. **Neural network mapping (NNM)** is a vital procedure in CIM design, which has a considerable impact on hardware performance. Mapping strategies such as quantization algorithms and data representation methods must be chosen carefully, considering accuracy performance and hardware overhead. In conventional CIM works, each design applies variations of the mapping strategies based on the designer's intuitions. However, these specific designs cover merely a small portion in a large design space of CIM design. Before developing circuit-level designs, a comprehensive, analytical, and accurate early-stage estimation would help designers explore the entire space instead of picking a specific approach with random attempts. This work demonstrates a system-level performance-focused design space exploration of NNM in CIM. A comprehensive joint hardware-software analysis is presented, providing a useful guideline while mapping DNN models onto CIM accelerators. The contributions of this paper can be summarized as follows:

- A complete design flow of mapping the neural network to the eNVM-based CIM architecture is presented. Under a precondition of maintaining accuracy performance, the design space of different quantization algorithms (fixed-point / dynamic fixed-point) and mapping strategies (2's complement / differential pair / shifted unsigned INT number representation) is comprehensively discussed.
- We explore the hardware requirements and evaluate the system-level performance of CIM accelerators for different quantization methods and mapping strategies. Statistics of input sparsity are compared, highlighting the impact of the input distribution on hardware metrics such as energy consumption.
- Based on our extensive algorithm-hardware joint analysis, we find that input sparsity is dissimilar according to different quantization methods, leading to different energy consumptions of CIM hardware. In general, we prefer higher input sparsity for better energy efficiency such as provided by dynamic fixed-point quantization. Besides, different mapping strategies also lead to different hardware performances. Our evaluations show that optimal design options such as 2's complement representation and shifted unsigned INT representation for the CIM system could improve the energy efficiency and throughput by

$2 \times$ and $1.2 \sim 1.6 \times$, respectively. The area overhead could also be correlatively reduced by 5%~25%. The baseline design employs fixed-point quantization and differential-pair number representation approach.

2 PRELIMINARY

2.1 CIM Basics

DNN processing is filled with massive convolutions, which can be mapped to 2D VMMs between the input vector and the flattened weight matrix. After mapping 2D weight matrices into memories properly, the CIM crossbar array structure can naturally support VMMs with perpendicular input rows and output columns. Various types of memory devices have been investigated for CIM designs. Among them, eNVMs are preferred for their non-volatile characteristic, multi-level capability, and dense cell structure. CIM designs generally leverage adjustments in peripheral circuits of the memory array to support in-memory computing. As shown in Figure 1, the input vector is loaded to the rows in parallel and multiplied by weights conductance to generate products in the form of current. By applying Kirchhoff's law, the summed current along columns represents the final MAC outputs. As this MAC operation is done in the analog domain, array-level interfaces such as **Analog-to-Digital Converters (ADCs)** and **Digital-to-Analog Converters (DACs)** are required between the memory array and other peripherals, offering inter-array or upper-level digital processing. This mixed-signal computing scheme offers the scalability towards hierarchical system designs via interconnect buses or **network-on-chip (NOC)** while introducing additional power dissipation and area overhead for the necessary data conversion.

For further reference in this paper, The term 'digit' will refer to inputs and weights used in the single MAC operation (as shown in Figure 1(a)) to differentiate them from the binary bits. In other words, one digit could represent a k -bit data. This digit-wise MAC itself could support multi-bit operation with high precision input digits encoded as analog signals by DACs and multi-bit weight digits stored in **multi-level cells (MLC)**. Multi-digit precision MACs could be further supported in CIM by encoding the high-precision inputs as pulses in sequence and splitting weights into multiple cells. Then the digitized MAC outputs by ADC will go through the shift-and-add process to obtain the final outputs. Since DNN models necessitate quantization before being mapped to CIM hardware, the quantized data precision will also affect ADC (& DAC) resolution accompanied with different hardware overheads. Besides, different data representation methods would cause different data distribution, leading to different memory capacity and ADC requirements.

In this work, as shown in Figure 1(b), we map the weights at different spatial locations of each kernel into different sub-matrices. For example, all the weights at the left-top channel of each kernel are restructured as one group (i.e., one sub-matrix). Hence, $K \times K$ sub-matrices are needed for mapping all the kernels. Considering the hardware constraints of memory array, a sub-matrix's size may still be too large. In this case, each sub-matrix can be further partitioned into a group of sub-arrays, and the input data can be assigned to corresponding sub-arrays. During the parallel computation, the segments of partial sums are first collected from sub-arrays, and then the partial sums from sub-matrices need to be accumulated to obtain the final output.

2.2 Low Precision Neural Network

Regardless of the hardware computational cost, today's DNN structures tend to grow larger and larger to improve software performance. This trend makes them unfriendly for edge devices, which have limited area and power budgets. The research efforts to solve this problem could be classified into two types [7]. The first one is to design compact network structures such as MobileNet [8] and ShuffleNet [9]. The second one is to quantize the large networks to reduce the computation and memory cost. The compact networks are not suitable for CIM-based inference engines for two

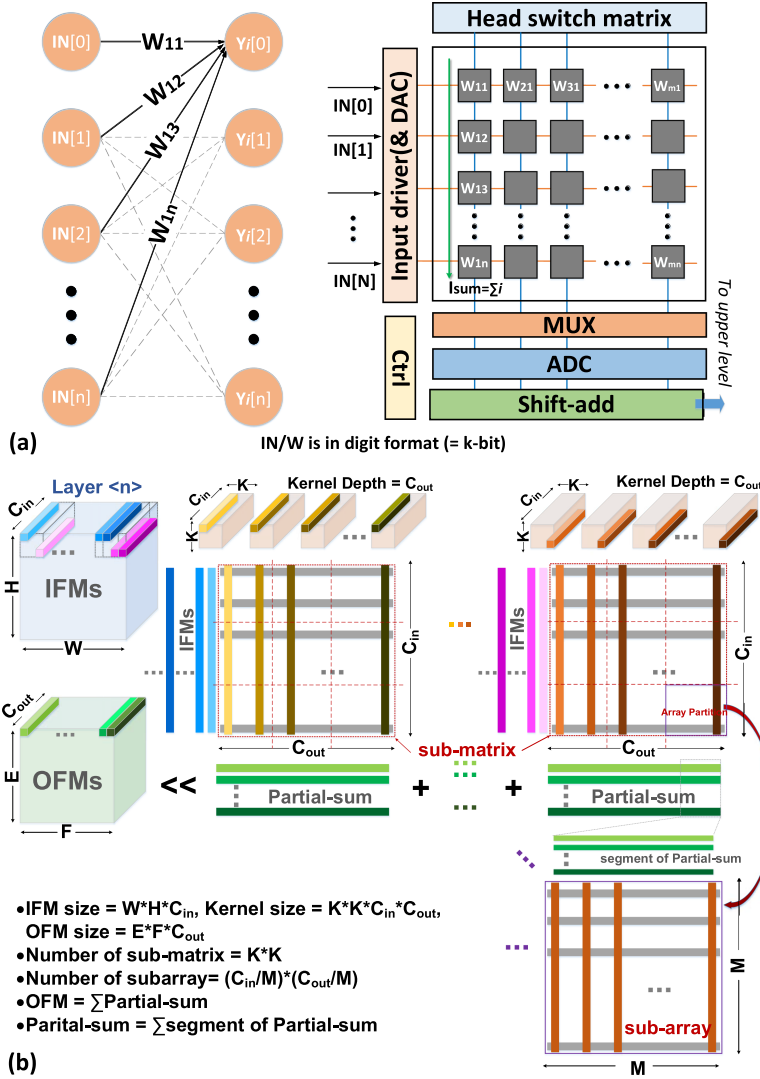


Fig. 1. (a) Generic structure of CIM array (b) Mapping of the convolutional layer to the CIM arrays.

reasons: one is that the compact networks still adopt floating-point operations, which are not easily mapped to the CIM array; the other reason is that CIM prefers the VMM operations with a big fan-in to open all the rows in parallel to improve the accumulation efficiency in the analog domain. However, in order to reduce the number of parameters and operations, the compact networks tend to adopt depth-wise convolutions, which have a small fan-in, thus are not efficient to be implemented with CIM structure.

Accordingly, most CIM engines are designed to support the networks with quantized parameters. The main focus of the network quantization is to map the high precision floating-point input/weight to the low precision fixed-point input/weight. Various works [7, 10, 11, 12, 13, 14] have proved that DNNs could be quantized to 1~8 bit weight/input with negligible accuracy loss. In this case, the VMM can be conducted with low precision fixed-point MAC unit, which is more energy and area efficient. In general, the lower the precision, the better the hardware performance

could be achieved. Thus, most quantized networks aim to reduce the precision of the parameters as much as possible. Approaches to obtain a quantized inference network could be sorted into two groups: 1) directly quantizing a well-trained floating-point network, and 2) quantization-aware training. The latter approach could achieve more aggressive precision reduction, with which binary input/weight have been proved applicable [13, 14]. However, these binary networks require special hardware design while having scalability issues towards more complex datasets. From the hardware's perspective, generality needs to be considered for a practical design. Thus, a quantization method with good scalability and flexibility will be preferred for the CIM inference engine. Various quantization methodologies could meet these requirements with a calm precision reduction [7, 10, 11, 12]. Then there are two basic questions to answer: which quantization method should we choose for the CIM inference engine? Will the quantization methods make a difference with varying parameter precision? The following part of this section will compare two typical quantization algorithms and discuss their properties from the hardware's perspective.

B. Jacob et al. [7] propose a general format that views the quantization as an affine map from the real value (Y) to the quantized value (X) as the Equation (1).

$$Y = C * (X + D) \quad (1)$$

Since X has lower precision than Y , it generally only represents a sub-range of Y . From the software's point of view to quantization, choosing this range is very important for good performance. It is usually harmless to directly quantize the weights into the $[-1,1]$ region. However, the inputs need to be taken care of during the quantization since they could vary substantially according to the network structure. Some methods use a fixed range across all the layers, which could be viewed as a fixed-point quantization. For example, WAGE [12] directly clips the input range to $[-1,1]$ with fixed constants for normalization between layers. This fixed range is achievable by controlling the statistics of the weights from initialization and **learning rate (LR)** to avoid exploding and vanishing. WAGE is hardware friendly since it only considers fixed range clips while no normalization is needed, which is usually in higher precision and costly. However, the scalability of WAGE is proved poor as the network goes deeper or the network structure becomes more complicated since it is hard to maintain the network's statistics only through initialization and LR. The upgraded WAGE [15] introduces **batch-normalization (BN)** (i.e., WAGEBN) into networks with a fixed range input quantization. BN could help stabilize the layer output statistics and make applying the fixed range quantization safer. Still, the BN output range is not necessarily limited, and thus may suffer from the overflow loss. Some other works utilize nonlinear functions like Tanh or Sigmoid to reform the output for fixed range quantization, requiring high-precision operation, and thus introducing larger hardware overhead. Another method is to find the quantization range from the input statistics dynamically. For example, [10] proposes a **dynamic fixed-point (DF)** number scheme to quantize the network for each layer, which could be viewed as dynamically finding the input (Y) range and mapping it to a fixed-point number set X by using layer-wise C .

This work will compare the hardware performance of these two quantization methods, namely WAGE and DF, as an illustration. It will be seen that even though the same precision is adopted, the quantization method will affect the data distribution and further change the hardware performance. Figure 2 shows the accuracy results of the quantization method on the VGG-8 network for CIFAR-10 classification, and the ResNet-18 network for CIFAR-100/ImageNet (subset) classification. WAGE performs well for VGG-8 but introduces a big accuracy drop when it moves to ResNet-18, thus BN is necessary for deep networks. Both methods could achieve the floating-point baseline accuracy with 8-bit input and 8-bit weight, and the accuracy degradation while further quantizing the weight to 2-bit is almost negligible, which is desired to hardware design. While these two algorithms are not **state-of-the-art (SOTA)** quantization algorithms that could achieve

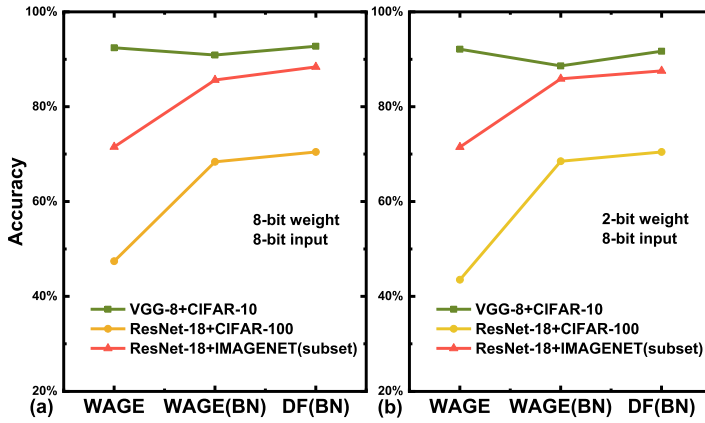


Fig. 2. Accuracy performance vs. network quantization approaches with (a) 8-bit weight/8-bit input (b) 2-bit weight/8-bit input.

the highest input and weight precision reduction, they are quite representative in deciding the quantization range. WAGE utilizes this fixed-point quantization which means the range is always fixed. The DF uses a dynamic range, so the cut-off effect of the input is much weaker. Many SOTA quantization methods could be viewed as an improved version of these two if we only consider the quantization range selection. For example, LSQ [16] introduces the learning step, which is quite similar to DF, but the step is learned while DF will use a step calculated by the maximum value for each step. The OIL [17] works more like WAGE with a fixed quantization range but uses a transformer to normalize the real value instead of a direct cut. The main difference between these two kinds of methods that will affect the hardware performance is whether the quantization has a hard-clipped result of the input.

While the quantization algorithms proposed by the software community attempt to reduce the input/weight precisions to a constant value for a general mapping to low precision hardware, the parameter precision could be further reduced considering the specific hardware design. For example, RaQu [18] processes the layer-wise quantization and further finetunes the bit width of groups of weights. Similarly, Zhenhua et al. [19] proposed the RRAM Computing Deviation Aware Quantization Scheme to search for the optimal precision for the weights and activations. These layer-wise or grainer group-wise quantizations could introduce a further precision reduction and potentially yield better hardware performance. However, this work mainly focuses on general cases to explore the desired properties of the inputs and weights instead of exploring the best quantization method for CIM. Thus, these advanced approaches are not covered in the discussion.

3 NEURAL NETWORK MAPPING

As aforementioned, most of the CIM architectures for DNN adopt a mixed-signal scheme that first conducts MAC operations in the analog domain. Then, the analog results are converted back to the digital domain for further processing, such as BN, ReLU, and pooling function. Besides, considering the limited size of the CIM array, it is usually hard to fit the whole weight matrix of one DNN layer into a single array. The weight matrixes are generally cut into several subarrays to generate partial sums. The partial sums are converted back to the digital domain for further accumulation. Thus, there are three key problems to solve when mapping the CNN to the CIM system: (1) How to map the MAC operations to the analog domain; (2) How to convert the analog MAC results back to the digital domain; and (3) How to perform the following digital processing. The design space of these three steps is large and a number of techniques have been

proposed. While the hardware implementations could differ from one design to another, the basic ideas behind them are common. This section illustrates some general design options and discusses the trade-offs between them. This work ignores the variability of dataflow mapping of 3D Convolutional operation to 2D MAC as we fixed this option as the one mentioned in Section 2.1. Other dataflow mapping methods could also be applied to improve the parallelism [20]. The three key steps are critical regardless of which dataflow mapping is used.

3.1 MAC Mapping in CIM

3.1.1 Number System in CIM. When mapping MACs to the CIM array, we need to make three decisions: the number system, the number representation, and the analog mapping method. Unlike digital computer systems, which are typically binary number systems, the CIM architecture employs analog calculation, thus could support high-precision digits in the MAC operation. As a result, different digit precision of input/weight may lead to different hardware requirements or overhead. For instance, increasing the input digit precision could reduce the compute cycles while expensive input-encoding circuits such as DACs need to be employed, causing a large area overhead. Generally, reducing the hardware cost is more critical than speeding up the computation for the resource-constrained edge devices. Thus, binary input digits are widely used in CIM designs. In contrast, increasing the weight digit precision could reduce the needed memory capacity by storing multi-bit value into one cell (e.g., 2-bit or 4-bit per cell) with negligible overhead as long as memory characteristics support multi-level states. Besides, as the digit precision of inputs or/and weights increases, the full precision of the analog MAC outputs will also increase, leading to higher required ADC resolution and overhead. Theoretically, increasing the digit precision for weights (cells) or inputs has the same impact on deciding the full precision of MAC outputs. The difference is mainly the hardware cost. Considering all the above, the number system we employed for CIM architecture discussed in this work will be binary inputs with flexible precision weights.

3.1.2 Number Representation in CIM. As the quantization ranges are symmetric around zero for these two demonstrated quantization methods (i.e., WAGE and DF), choosing N-bit **integer (INT)** as the X for the quantized fixed-point number is natural to keep consistency across different quantization methods. The quantized inputs and weights of the convolution/fully-connected layer could be easily converted to INT format with a scalar C , as shown in Equation (1). In the traditional digital computer, the 2's complement scheme is the most commonly used method for integer's representation and calculation, which is also a straightforward choice for CIM systems. Despite that the 2's complement representation could encode the sign in the binary sequence for MAC operations in CIM, sign extension is necessary but inefficient to obtain the correct result. Alternatively, as discussed in our prior work [21], we view the 2's complement data calculation as a weighted sum of the power of 2 bases in the CIM system, whose weights are unsigned digits. The 2's complement number system could encode positive and negative numbers in a binary number representation as in Equation (2). Unlike the digital MAC operation, which performs accumulation after full precision multiplication is done, the CIM design performs the digit-decomposed MAC operation that first processes digit-wise MAC and then is followed by shift-add operations to restore the significance information. In other words, the bit-wise multiplication (AND) result b_n of different rows will be accumulated first in an unsigned format. Then, this bit-wise partial sum will be scaled by the base 2^n , which is shift operation in the binary domain, and accumulated to the partial sum. Based on the sign of the scaler, an additive inverse operation may be applied before the shift.

$$x = b_{N-1} \cdot (-2^{N-1}) + \sum_{n=0}^{N-2} b_n \cdot 2^n \quad (2)$$

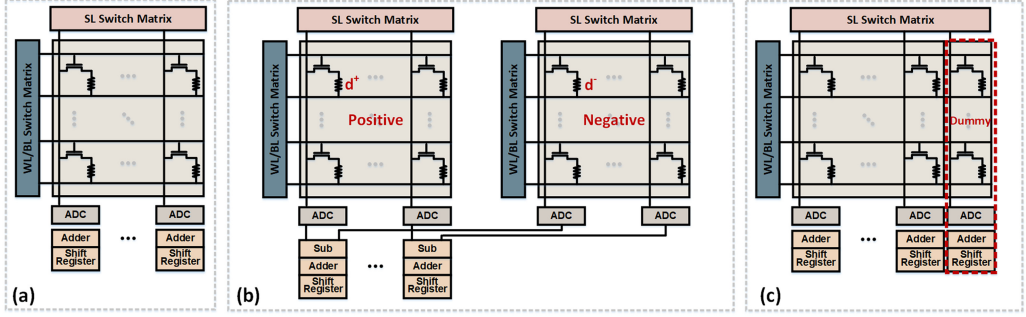


Fig. 3. Subarray structures for different mapping (data representation) methods: (a) Case1: 2's complement extended representation (b) Case2: differential-pair data representation (c) Case3: shifted unsigned INT.

This format could be extended to arbitrary digit precision with the power of 2^k bases by grouping k -bit together. Equation (3) shows an illustration of a 2-bit digit representation. In this case, two adjacent binary bits b_{2n+1}, b_{2n} could be combined as one 2-bit digit $d_n \in [0, 1, 2, 3]$ as shown in Equation (4). The base will be the same as the lower precision bit, which is b_{2n} in this specific case. It is worth noticing that the combination could only be applied to two adjacent bits with the same sign. For the most significant bit b_{N-1} , it could not be grouped with its neighbor b_{N-2} .

$$b_{2n+1} \cdot 2^{2n+1} + b_{2n} \cdot 2^{2n} = (b_{2n+1}b_{2n}) \cdot 2^{2n} = d_n(2^2)^n \quad (3)$$

$$x = b_{N-1} \cdot (-2^{N-1}) + b_{N-2} \cdot (2^{N-2}) + \sum_{n=0}^{\frac{N-2}{2}-1} d_n(2^2)^n \quad (4)$$

Thus, for an N -bit value represented by k -bit digits in this format, $\lceil (N-1)/k \rceil + 1$ terms are needed in total, as shown in Equation (5). b_{N-1} could also be represented by a k -bit digit, but its value is only 0 or 1. We call this 2's complement extended representation method as Case1 for easy reference later. The array structure needed for this representation is shown in Figure 3(a). Case1 data representation have been adopted in CIM designs [22, 23].

$$x = b_{N-1} \cdot (-2^{N-1}) + \sum_{n=0}^{\lceil \frac{N-1}{k} \rceil - 1} (d_n) \cdot (2^k)^n, \quad d_n \in [0, 1, 2, \dots, 2^{k-1}] \quad (5)$$

Since the sign bit will cause potential issues if we want to extend the two's complement to the high-precision digit representation, another way of data representation in the CIM is to use two unsigned digits as a differential pair to represent signed value, which is referred as Case2. The digit representation of this method could be viewed as the Equation (6). If the weight is positive, the d_n^+ will represent its value while the d_n^- will be zero. Otherwise, the d_n^+ will be zero while the d_n^- will represent its absolute value. The number of pairs, in this case, will be $\lceil (N-1)/k \rceil$. While the number of pairs looks smaller than the two's complement case, the number of terms is twice of the number of the pairs, thus introducing a big overhead when k is small. A basic array structure of Case2 is shown in Figure 3(b). This differential-pair data representation is frequently used in CIM design such as [5, 24].

$$x = \sum_{n=0}^{\lceil \frac{N-1}{k} \rceil - 1} (d_n^+ - d_n^-) \cdot (2^k)^n, \quad d_n^+, d_n^- \in [0, 1, 2, \dots, 2^{k-1}] \quad (6)$$

Another solution for the sign bit is to avoid negative values in the mapping by using the unsigned INT as X with proper shift D (shown in Equation (1)). The number in the unsigned INT could be

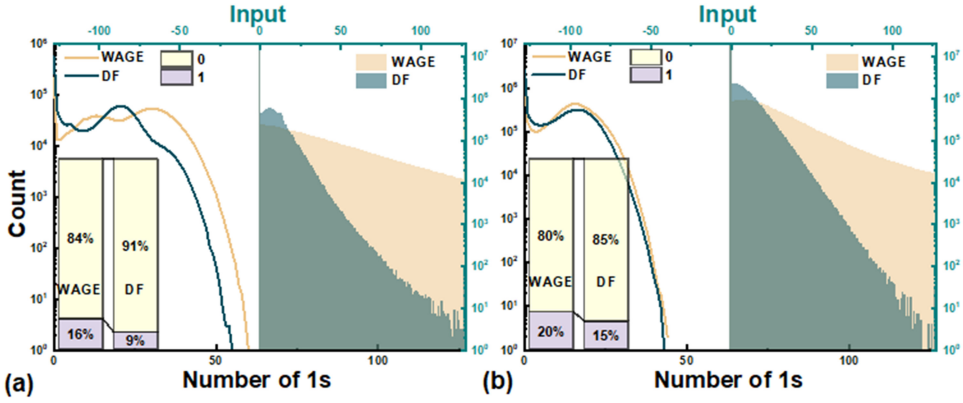


Fig. 4. Input distribution of different quantization methods on (a) VGG-8 network and (b) ResNet-18 network.

represented as shown in Equation (7). In this case, the number of terms for the high precision extension will be $\lceil N/k \rceil$. It is the smallest among these three methods for $k > 1$. This shifted unsigned representation is referred as Case3. To map the weights to the unsigned INT for Case3, a positive shift D_w is needed to map the weight W^r to X_w^r Equation (8), where the superscript r denotes the weight from different rows. Thus, the MAC operation between the weights and inputs will be calculated as shown in Equation (9). An additional MAC term will be introduced between the inputs and the shift. Figure 3(c) shows that a dummy column of D_w is needed for Case3 to match the term $\sum_r D_w \times IN^r$. Reference [4] has adopted this scheme for the CIM array design.

$$x = \sum_{n=0}^{\lceil \frac{N}{k} \rceil - 1} d_n \cdot (2^k)^n \quad (7)$$

$$X_w^r = W^r + D_w \quad (8)$$

$$\sum_r (X_w^r - D_w) \times IN^r = \sum_r X_w^r \times IN^r - \sum_r D_w \times IN^r \quad (9)$$

As aforementioned, we set the input digit in this paper in binary format, which is naturally compatible with 2's complement representation with simple hardware implementation. In other words, inputs always utilize the Case1 mapping strategy while weight mapping methods will be evaluated from Case1 to Case3, respectively. These three mapping strategies could have many extensions in real implementations. For example, Wan et al. [25] use a differential pair in their weight encoding, but instead of dividing the positive and negative to separate arrays, the subtraction is directly done in the adjacent cells using inputs with opposite orientations. Jiang et al. [26] utilize a current mirror to implement the subtraction in the analog domain before ADC. Again, this work focused on the basic design space options instead of these improvement by detailed circuit design techniques.

3.1.3 Hardware Implementation. In the circuit implementation, the binary inputs are encoded as 0V and read voltage, representing "0" and "1", respectively. As the 0V input will not contribute current to the partial sum and thus consume no energy consumption, the CIM could automatically skip the zero digit in the input. We found that the sparsity of the input will be directly affected by the quantization method, as shown in Figure 4. Due to the hard-clip manner of the WAGE, the inputs of WAGE are wider spread than DF and thus have fewer "0" input digits. Thus, while different quantization methods could achieve similar software performance under the same precision, their input statistics could be dissimilar, affecting the real hardware performance. The small digits

will be mapped to low conductance values for the weight encoding, contributing smaller currents than big conductance. Thus, different weight digits will also cause different energy consumption, and small digits are preferred for high energy efficiency. However, as we mentioned in Section 2, we always quantize the weights to a fixed $[-1, 1]$ range as it causes not much difference across different quantization methods, the statistics of weights are similar for WAGE and DF. Thus, the hardware performance is mainly affected by the input statics difference instead of weight statics across different quantization methods in this work. Other techniques such as pruning [27, 28] or training with a regularizer [29] may further improve the energy efficiency from the weight side.

When mapping a k -bit digit to a k -bit eNVM cell, the cell's conductance is divided into 2^k levels between G_{min} and G_{max} as shown in Equation (10).

$$G_n = d_n \times \Delta G + G_{min}, d_n \in [0, 2^k - 1], \Delta G = \frac{G_{max} - G_{min}}{2^k - 1} \quad (10)$$

In this case, “0” in the k -bit digit is mapped to a nonzero conductance G_{min} and the digit levels to the conductance mapping is a shifted scaling. Thus, considering the digit-wise analog MAC operation of the i^{th} digit of input and n^{th} digit of weight across different rows (r) as shown in Equation (11), there will be an input-dependent output shift $\sum_r V_i^r G_{min}$ in the eNVM-based CIM structure.

$$I_{i,n} = \sum_r V_i^r (d_n^r \times \Delta G + G_{min}) = \sum_r V_i^r (d_n^r \times \Delta G) + \sum_r V_i^r G_{min} \quad (11)$$

This input-dependent output shift is undesired since the analog MAC output will vary with input patterns, leading to different summed $I_{i,n}$ for the same ideal MAC value, which is not friendly to ADC design. When the on/off ratio of the cell is high and the cell precision is low, the current contributed by the shift term is negligible with no accuracy drop. However, if the on/off ratio is limited or the cell precision is high, the current caused by this shift term will make the analog output of different partial sums overlap, harming the accuracy performance. Luckily, according to the mapping method, this input-dependent output shift could be canceled to some extent. Case2 and Case3 mapping methods could naturally cancel this G_{min} in digit-wise analog MAC level. As shown in Equation (12), as the weights are represented by differential pairs (Case2), the shift term will be canceled among the positive and negative digits.

$$I_{in} = \sum_r V_i^r (d_n^{r+} \times \Delta G + G_{min}) - \sum_r V_i^r (d_n^{r-} \times \Delta G + G_{min}) = \sum_r V_i^r (d_n^{r+} \times \Delta G) - \sum_r V_i^r (d_n^{r-} \times \Delta G) \quad (12)$$

Similarly, as the dummy D also has the shift term in the Case3 approach, the input-dependent output shift term could also be canceled, as shown in Equation (13).

$$I_{ij} = \sum_r V_i^r (d_n^r \times \Delta G + G_{min}) - \sum_r V_i^r (d_{-D_n}^r \times \Delta G + G_{min}) = \sum_r V_i^r (d_n^r \times \Delta G) - \sum_r V_i^r (d_{-D_n}^r \times \Delta G) \quad (13)$$

Although there is no straightforward approach to cancel the shift on digit-wise analog MAC level for Case1, the shift-add operation across different weight digits could cancel the G_{min} term to some extent. As shown in Equation (14), since the input is the same across different weight digit, the shifts across them are also the same. It could be viewed as the shifts being scaled by the sum of the bases, which will be “-1” in the binary case. This means the shift of different significant bits will cancel each other and leave a relatively small shift for the shift-added result. As the weight

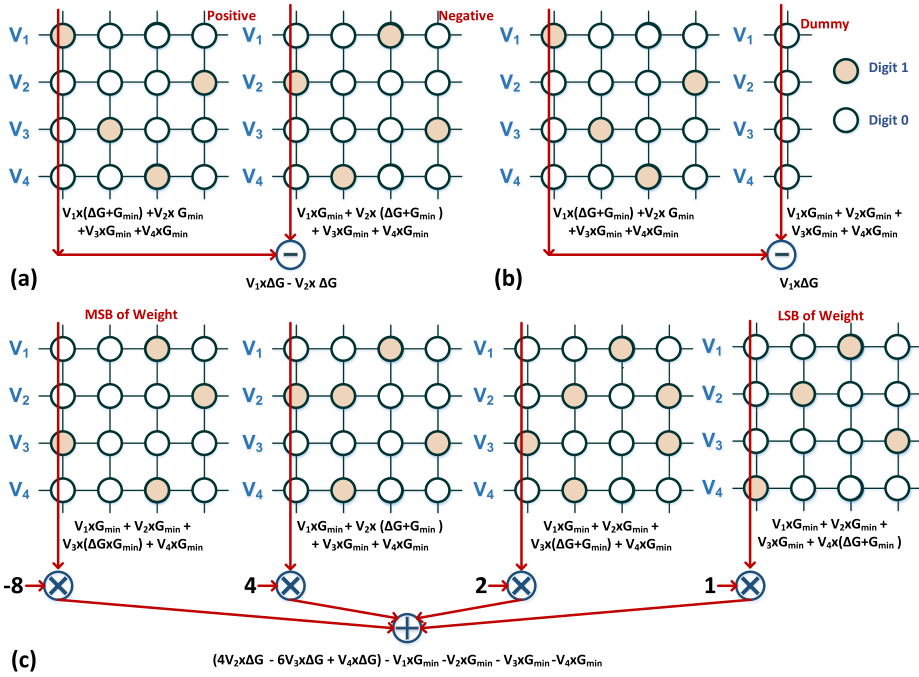


Fig. 5. Hardware mapping with input-dependent cancellation for (a) Case2: differential-pair data representation; (b) Case3: shifted unsigned INT; (c) Case1: 2's complement extended representation for binary digit.

digits' precision increases, such cancellation will be less effective. At this point, a dummy column of all- G_{min} cells could be used to cancel the effect.

$$I_i = I_{iN-1} \cdot (-2^{N-1}) + \sum_{n=0}^{N-2} I_{ij} \cdot 2^j \quad (14)$$

$$I_{i_{shift}} = \sum_r V_i^r G_{min} \cdot (-2^{N-1}) + \sum_{n=0}^{N-2} \left(\sum_r V_i^r G_{min} \right) \cdot 2^n = \sum_r V_i^r G_{min} \left(\sum_{n=0}^{N-2} 2^n - 2^{N-1} \right) = - \sum_r V_i^r G_{min} \quad (15)$$

Examples of shift cancellation for different mapping methods are shown in Figure 5 with a 4-row accumulation and 4-bit weight with binary digits. It should be noted that cancellation methods here ignore the ADC quantization impact for shift cancellation. While the ADC is taken into consideration, the shift may not be able to be totally canceled because of the introduced quantization error. However, the remaining shift will be small under proper ADC precision.

3.2 Analog-to-Digital Conversion

In most of the CIM designs, the conversion from the analog to the digital domain is the bottleneck of hardware performance. In general, high parallelism is preferred to improve the efficiency of the analog MAC, which will increase the full precision of the partial sum. For lossless conversion, a high-resolution ADC is essential. However, the high-resolution ADC is usually area-consuming, introducing difficulties to match the column pitch of the array. In this case, several columns need to share one ADC through a multiplexer, which reduces the column-wise parallelism. In addition, the limited dynamic range also causes challenges in high-precision ADC design. Besides, as illustrated in previous work [30], the ADC contributes a large portion of total energy consumption and latency in CIM design, becoming more severe with increasing ADC precision. There are mainly two

strategies used in analog output digitization. The first is to limit the number of rows open in parallel [31], which only needs low-precision ADC for no quantization loss. This method will relieve the dynamic range problem and fit more ADCs to the column pitch. However, this will not improve the throughput since it increases the column-wise parallelism by sacrificing row-wise parallelism. Another drawback is that energy efficiency will be limited as more cycles of low precision quantization are conducted. The advantage of this method is that the full precision conversion could guarantee the software's performance. The second method is to introduce ADC quantization loss during the conversion. Designing the ADC according to the partial sum distribution is important to minimize the performance loss with the highest precision loss. For example, Reference [6] shows that instead of cutting LSB for the linear quantization in ADC as the general quantization method, it will be more efficient to cut MSBs of the partial sums as they are around "0"s in most cases. Thus, partial sums' MSBs and/or LSBs are cut according to their distribution for the linear ADC quantization. To further fit the ADC quantization to the distribution, Reference [32] uses the Lloyd-Max algorithm to find nonlinear references for quantization. Reference [29] uses the **Cumulative distribution function (CDF)** of the partial sum to decide the nonlinear references corresponding to linear quantized cumulative distribution. No matter how these nonlinear references are decided, the ADC levels must be mapped to these references through a **look-up table (LUT)** for further processing. Also, compared to the linear quantization, the nonlinear quantization is more statistics dependent. This work chooses the former one as the case study for the nonlinear quantization. Introducing quantization loss could release the dynamic range and maintain parallelism with reduced energy consumption and latency from ADC. However, the quantization method combined with the MAC mapping method could change the partial sum distribution even for the same task. As the ADC precision reduction heavily depends on partial sum distribution, we need to explore the best point for each task on a different design.

3.3 Post-ADC Digital Processing

As mentioned before, the weight matrixes are partitioned into different subarrays, and the MACs are digit-wise. Thus, the MACs from different precision need to be shift-added across different significant digits, and the partial sums from different subarrays are added together using pure digit circuits. These digit circuits are assumed to be fixed-point whose precisions are decided by the ADC output. The ADC output precision will be the same as ADC precision for the linear quantization. However, for the nonlinear quantization, the LUT will give a higher-precision ADC output to represent the nonlinear levels, thus introducing higher hardware overhead for the digital circuit. The final outputs of the Conv/FC layers will be high-precision fixed-point values. According to the network structure or quantization method, some intermediate functions, such as BN and/or scaling, need to be applied between two layers of Conv/FC. Some works have proposed using low precision operations for these functions with some modifications in the flow [11, 12]. Since this work mainly focuses on the CIM structure, we use a **floating-point unit (FPU)** to support these operations. We also assume that if there are back-to-back BN and Scaling, they will be merged into one FPU operation. This assumption will not affect the system-level performance as the computations of Conv/FC layers dominates the hardware performance.

4 EVALUATION RESULTS

In this section, we evaluate the hardware performance under different quantization and mapping options of VGG-8 for CIFAR-10 classification and ResNet-18 for CIFAR-100 and ImageNet classification. All the networks are quantized to 8-bit inputs with 8-bit weights. The input digits are always 1-bit, while the weight digits vary from 1-bit to 4-bit. We first verified the accuracy performance of different weight digits under with varying memory on/off ratio and then evaluate

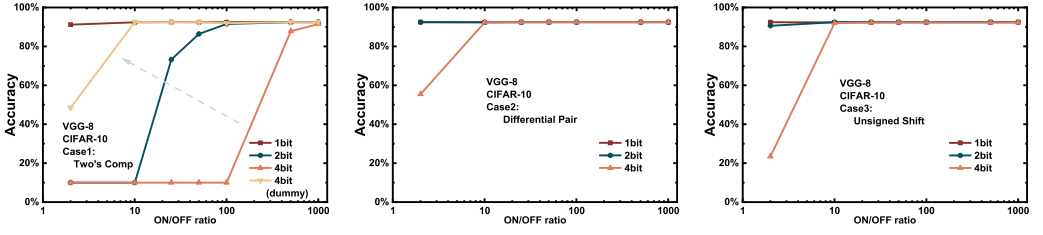


Fig. 6. Accuracy performance vs. on/off ratio with (a) Case1: 2's complement extended representation (b) Case2: differential-pair data representation (c) Case3: shifted unsigned INT.

the hardware performance with different design options with no ADC quantization loss (i.e., full-precision SAR-ADC is hired). Finally, we explore the design space of CIM designs with reduced ADC precision to further improve hardware performance.

4.1 Hardware Trade-offs Among Different Mapping Strategies

As discussed in Section 3, different mappings will have different G_{min} cancellation abilities. Figure 6 shows the accuracy performance of three described number representations under different memory cell on/off ratios of the VGG-8 network with 1/2/4-bit weight digits. As illustrated in Figure 6(c), Case1 with the 1-bit per cell setting could maintain the accuracy even with a very small on/off ratio. However, the robustness disappears as the cell precision goes high as we analyzed. For the 4-bit cell precision, it even fails under a relatively large on/off ratio of 100. Thus, for the high-precision cell utilized in the Case1 mapping, we need either a very large on/off ratio or an extra all-zero column to cancel G_{min} . It could be seen that the 4-bit with dummy could recover the performance a lot. What can be clearly seen in Figure 6(b), (c) is that, since Case2 and Case3 could automatically cancel the G_{min} , the accuracy could be maintained across different precision settings with a larger on/off ratio range.

We then evaluate the hardware performance across different design options with full precision ADC. All the hardware performances are evaluated at 22nm node in DNN+NeuroSim V1.3 [33], which has been validated with real chip data. It is an open-source simulator consisting of different abstraction levels and analytical modeling for the memory cell and transistor technology. We use it to build CIM-based architectures with traces from real networks to evaluate the overall hardware performance. The RRAM devices' Ron/Roff are assumed to be 6k Ω /900k Ω [34]. The array size of the VGG-8 is assumed to be 128 $\times$ 128, while the ResNet-18 has an array size of 64 $\times$ 64. We choose the array size to be the minimum kernel *input channel depth* $\times$ *output channel depth* of the network (except for the first layer) for better memory efficiency. In this assumption, the hardware performance differences are purely caused by different quantization or mapping methods. According to the results shown in Figure 6(a), we employ a dummy column for Case1 to cancel the G_{min} with 4-bit cell precision. The trends of the chip area, energy efficiency, and throughput are shown in Figure 7. While comparing the performance of WAGE and DF with all the other design options consistent, we also find that the quantization algorithms make no differences on the area overhead/throughput. The reason is that the hardware for Conv/FC layer computation dominates the chip area/processing time. What is surprising in this comparison is the TOPS/W differences between DF and WAGE. DF reports much better energy efficiency than WAGE because there are more "0"s in the DF's input bits, which will not consume power. Thus, we could conclude that DF quantization is a better choice than WAGE considering better energy efficiency with full-precision ADC.

From the plots in Figure 7(a–c), it is apparent that, as Case2 requires two memory arrays to represent one weight digit array, its area is much bigger than the rest two for 1-bit cell. Case3 is slightly bigger than Case1 due to the additional dummy column. As the cell precision increases,

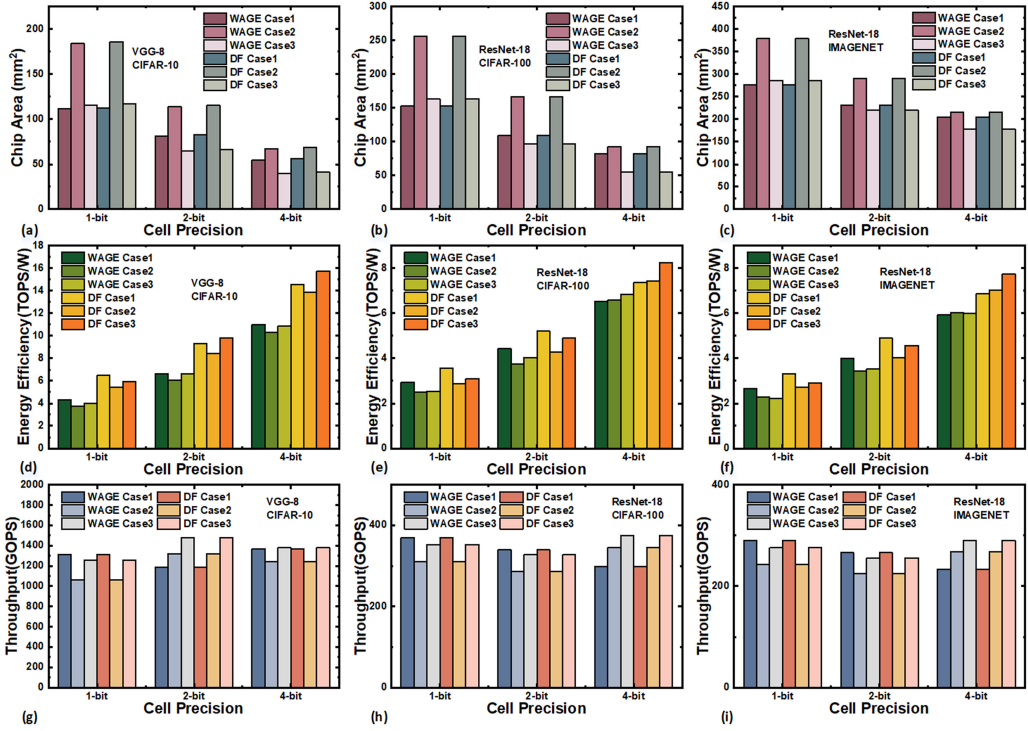


Fig. 7. Hardware performance (energy efficiency, throughput, area overhead) vs. different design options (quantization methods, mapping methods, cell precision) with no ADC quantization loss.

the difference gradually decreases. In general, Case2 is dominant on area overhead, even considering that Case1 has one extra array for the sign bit. For the 4-bit cell precision, as the Case1 need to introduce an all-zero dummy column for G_{min} cancellation, its array level structure will be the same as Case3. Nevertheless, Case3 requires the smallest size since it needs the least number of arrays for each weight array. On the other hand, as shown in Figure 7(d-f), while comparing energy efficiency among data representation methods, Case1 gives the best energy efficiency for the 1-bit cell precision while Case3 performance will catch up as the cell precision goes high. This is because, as the cell precision goes high, Case1 also needs to have a dummy column for G_{min} cancellation while it has one more array per weight for sign bit than Case3. For the Case2, the energy consumption is not double of the other two cases like the subarray area. This is because even though two arrays are used for one digit, one of them will be zero and consume relatively low power. We also observe that the trend of throughput has no clear trend as shown in Figure 7(g-i). The reason is that, unlike the area and energy cost dominated by in-memory computing, the CIM system's latency mainly consists of the array latency and the **network-on-chip (NOC)** latency. The array latency is highly affected by the peripheral designs such as ADC while the NOC latency is jointly determined by the number of bits to transmit in total, the wire length, and interconnection complexity. As the cell precision increases, the wire length and interconnection complexity tend to be reduced, benefitted from smaller chip size, while ADC resolution and output precision (determining the number of bits for data transfer) will increase, requiring longer processing time. Thus, system throughput has no obvious trend across design options. Under the same quantization approach, low cell-precision prefers the Case1 mapping method while high cell-precision prefers Case3 considering the energy efficiency and area overhead.

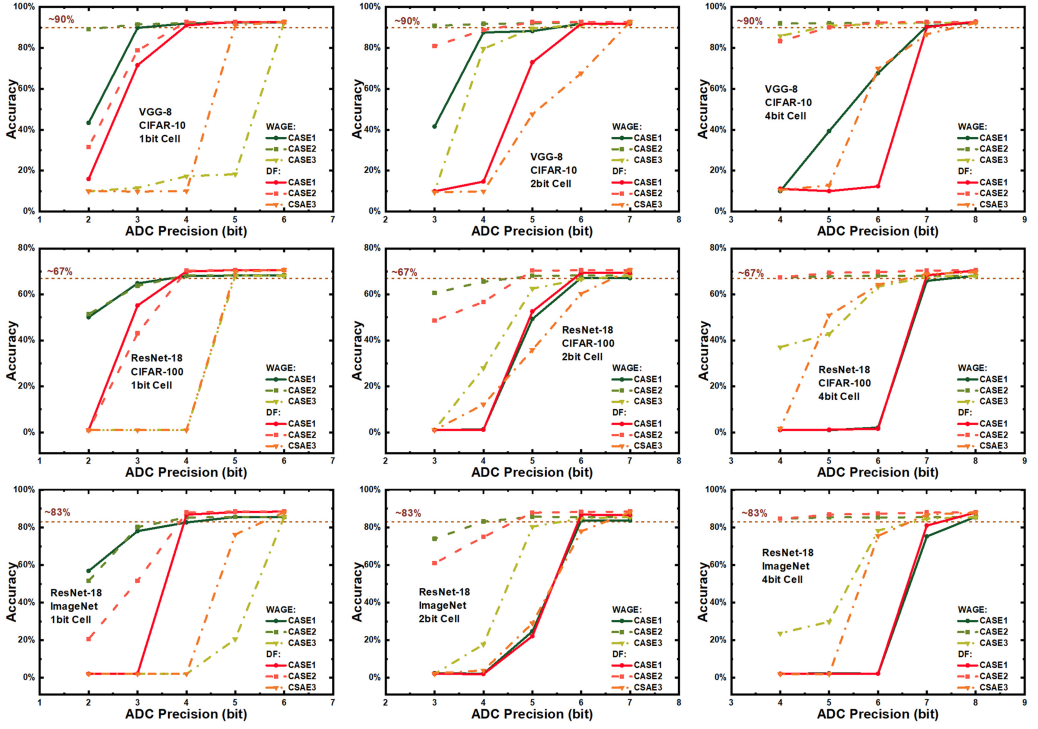


Fig. 8. Accuracy performance vs. ADC precision for different design options (quantization methods, mapping methods, cell precision).

All these findings can help us to find optimal design options for neural network mapping. Based on the evaluation results presented in Figure 7, the optimal design option (DF+Case3 for high cell precision and DF+Case1 for low cell precision) could achieve 29%~45% improvement on energy efficiency, 4%~40% area reduction, and 4%~25% speedup, compared with inefficient design options under the same cell precision.

4.2 Hardware Trade-offs with Different ADC Configurations

In the CIM designs, ADC plays an important role in hardware performance. Although the full-precision ADC could guarantee the network's software performance, it is not an optimal choice as the partial sum distributions show that full-precision ADC is overqualified. By examining the accuracy performance, required ADC precision could be lowered with tolerable accuracy loss. Since the accepted ADC quantization is mainly decided by partial sum distributions, which could be impacted by the network, task, quantization method and weight mapping methods, the lowest ADC precision needs to be checked with different design options. We assume the whole network utilizes the same ADC typology with the same reference levels for a simple design.

Figure 8 shows the accuracy performance across different ADC precision. The comparison among different quantization methods and different cell precision is also presented. By comparing different mapping strategies with the same quantization method, we find that Case2 shows the highest accuracy robustness to varying cell precision. The reason behind this is that the differential pair has at least one "0", which means the sparsity of either array is larger than the other two cases. Besides, the weight distribution across different significant digits has no significant differences in Case2. Case1 and Case3 have the problem of using the same ADC to cover two different distributions. Case3 works poorly with the low-precision cell since the data pattern of the dummy column

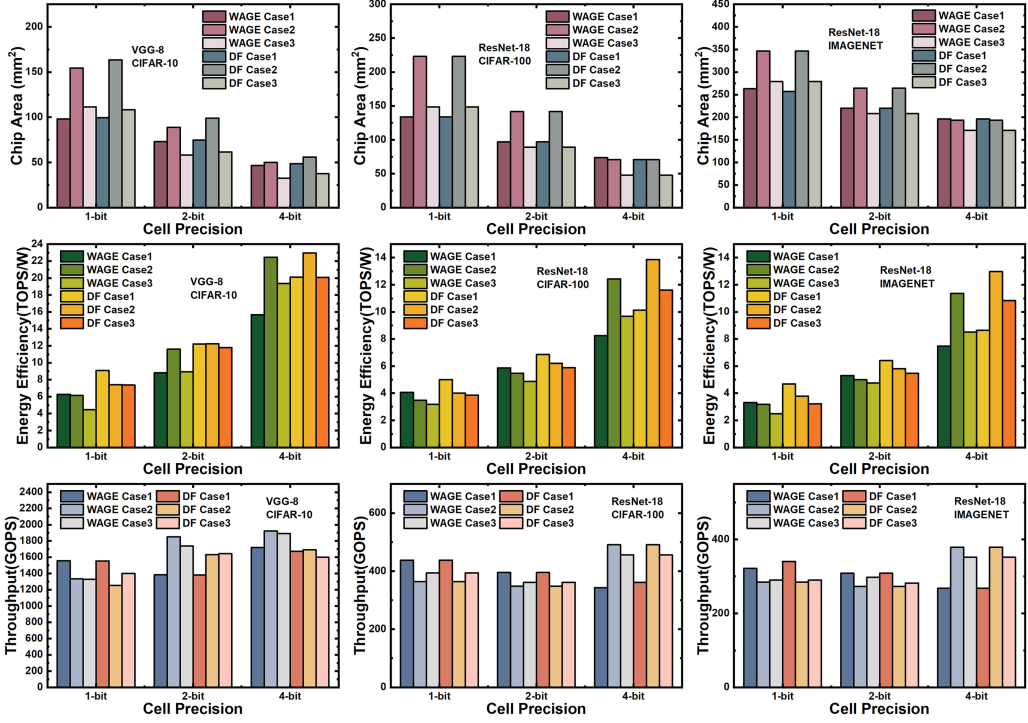


Fig. 9. Hardware performance (energy efficiency, throughput, area overhead) vs. different design options (quantization methods, mapping methods, cell precision) with tolerable ADC quantization loss.

is different from weight columns. Thus, the partial sum of the dummy will have a wider distribution than that of the weight array. Case1 works poorly with the high-precision cell since the sign bit is binary value while other digits are in high precision. Thus, the partial sum distribution of the sign column will be much narrower than that of the weight array. All three representation methods could tolerate similar ADC quantization loss with low-precision cell case (1-bit), while Case2 dominates with higher cell precision. In conclusion, Case2 could tolerate the most aggressive ADC loss across all cell precisions. As aforementioned, the ADC precision reduction will be bounded by the different distributions from segments or digits. In principle, improvements could be achieved by adjusting the ADC configuration or the distribution. For example, Yue et al. [35] propose the reconfigurable ADC, which could adjust the precision during calculation. Wan et al. [25] utilize the model-driven chip calibration technique method to finetune the partial sum to fully utilize the ADC range. These improvement methods are not the focus of this work but could be explained by the presented observations.

Next, we evaluate the hardware performance with the lowest ADC precision for each case that could maintain 90%, 67%, and 83% for CIFAR-10, CIFAR-100, and ImageNet (subset), respectively. Figure 9 shows the hardware performance across different quantization and mapping methods with the optimized ADC precision. We could see that the trend for the area overhead across different design options is quite similar to the full precision ADC case. However, as the cell precision goes higher, the area overhead of Case2 decreases because of reduced ADC overhead. For the low-precision cell, Case1 is still the best choice considering energy efficiency. On the other hand, as the cell precision increases, Case2 benefits from the aggressive ADC precision reduction, showing better energy efficiency. Because of the ADC saving from the mapping methods, Case2 of WAGE

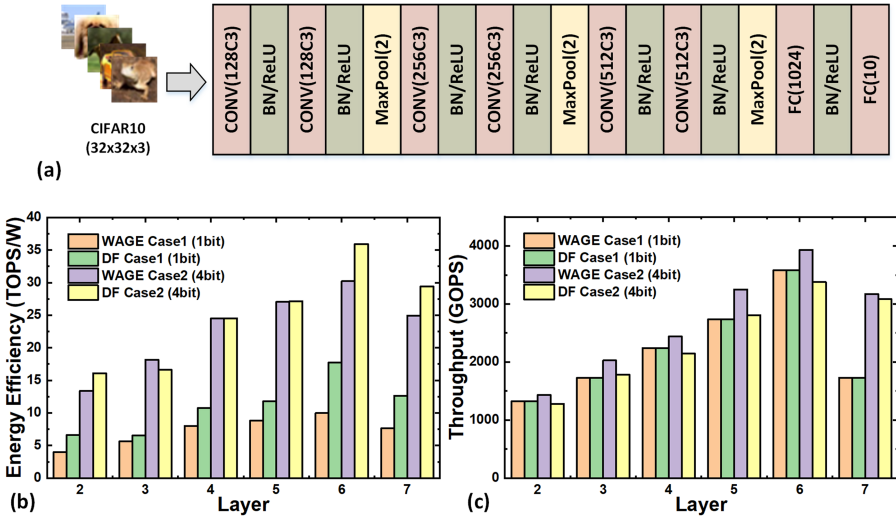


Fig. 10. Hardware performance (energy efficiency, throughput) vs. different layers of the VGG8 network.

tends to perform better than Case1/Case3 of DF even under the lower input sparsity, indicating that the combination of different design options collectively determines the energy efficiency. Similar to evaluation results of CIM design with the full precision ADC, there is no obvious trend for the throughput performance as the reduced ADC precision across different design options is quite different, making the chip area and the number of bits for transmission different.

We further evaluate the layer-by-layer performance for the VGG8 network consisting of six **convolutional layers (Conv layer)** and two **fully-connected layers (FC layer)** on CIFAR10 dataset classification as shown in Figure 10(a). Here the energy efficiency and throughput for the first and the last layer are not included since they have low memory efficiency due to small input or output channel depth, leading to non-typical results. Case1 mapping is evaluated for 1bit cells since it has the best hardware performance when the cell precision is low. The Case2 is evaluated for 4-bit cells with ADC loss introduced for the same reason. As shown in Figure 10(b), we could observe that the DF is better than WAGE in most layers considering energy efficiency. The performance across different layers varies since the input statistics of each layer are different. As the Conv layer goes deeper, the input size shrinks while the weight array size increases. As a result, the CIM array's energy becomes more dominant, showing the higher energy efficiency for the deeper layer. For the FC layer, it has less weight reuse than the Conv layers. Thus, although the CIM array energy is also dominant, FC layer performs worse than deep Conv layers but better than shallow ones. Figure 10(c) presents that the GOPS higher throughput is achieved for deeper Conv layer because of the dominated CIM operations over digital peripheral calculations. The input reuse decreases significantly for the FC layer compared to the 3×3 Conv layers. Thus, the IC delay is more dominant for the FC layer than the Conv layer with a similar weight array size, causing a decrease in throughput. In summary, the Conv layers with a small input size but a big weight size will achieve better hardware performance. For the similar weight size, the FC layer performs worse than the Conv layer because it only performs a one-time operation, and thus lacks data reuse.

As the ADC precision is critical to the hardware performance, some works have proposed using nonlinear ADC quantization to decrease the ADC precision further [32]. In general, the Lloyd-Max algorithm could be used to find the nonlinear quantization levels from the statistics of the partial sums. Traditionally, partial sums from all network layers are sampled together to find a global

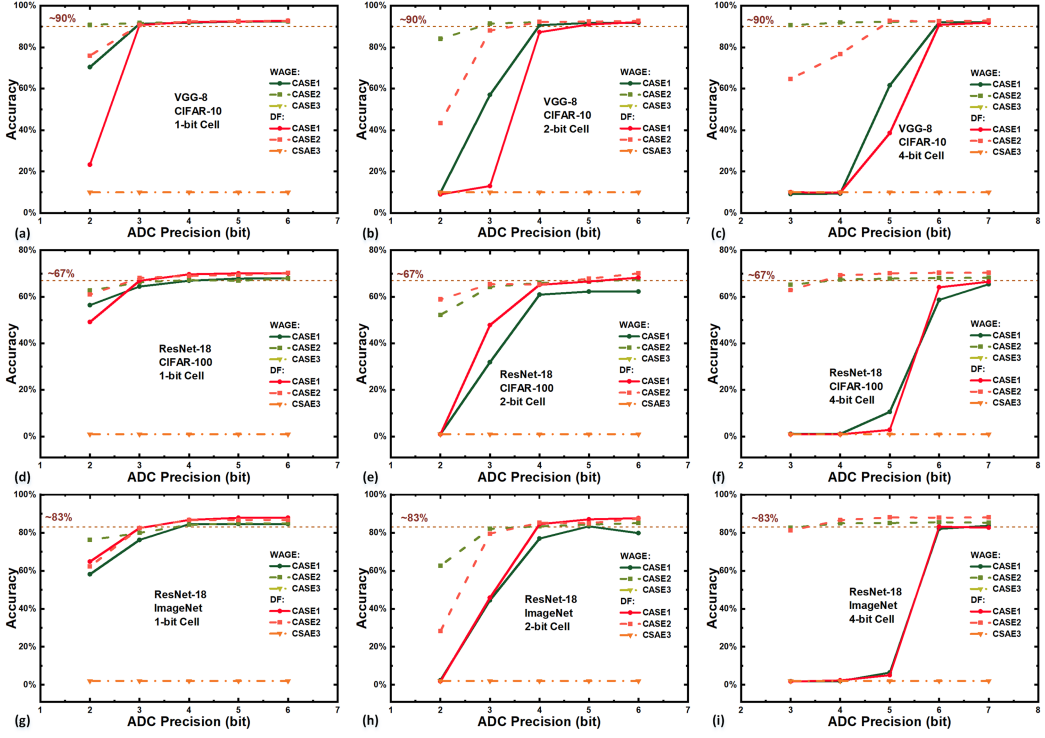


Fig. 11. Accuracy performance vs. ADC precision with nonlinear quantization for different design options (quantization methods, mapping methods, cell precision).

nonlinear quantization. However, a clear benefit of nonlinear quantization in the improvement of hardware performance could not be identified in our analysis. Figure 11 presents the evaluation results on accuracy performance with nonlinear ADC quantization. Comparing the differences between nonlinear quantization (Figure 11) and linear quantization (Figure 8), the accuracy of Case1/Case2 methods drops slower with nonlinear quantization than the linear quantization as ADC precision decreases. One unanticipated finding is that the nonlinear quantization does not work for Case3. This inconsistency could be attributed to the hired dummy columns to get the correct signed partial sum for Case3 mapping. Since the dummy column is shared among all the weight columns contributing negligible impact to the global statistics, nonlinear quantization levels will be ignorant of dummy statistics, causing big quantization errors. A potential solution could be assigning different ADCs for dummy and partial sums, which will increase the hardware overhead for reference generation on-chip. Compared to linear quantization, another notable finding is that there are some initial accuracy losses of nonlinear quantization while ADC precision starts to decrease from full precision (e.g., the accuracy performance shown in Figure 11(e) from 6-bit to 4-bit). This discrepancy is likely related to the exploited Lloyd-Max algorithm to find the nonlinear quantization levels. When the uniform nonlinear levels are applied to all the layers, the quantized partial sum may alter the following layers' input statistics because of the DNN's layer-by-layer structure, making the nonlinear quantization levels no longer optimal. These observational findings indicate that nonlinear ADC quantization could not guarantee software performance across different tasks, and the reduced ADC precision by nonlinear quantization is not encouraging. Additionally, extra hardware would be introduced for nonlinear quantization. For instance, a LUT array is essential to remember the digital values for analog nonlinear references to perform post-ADC

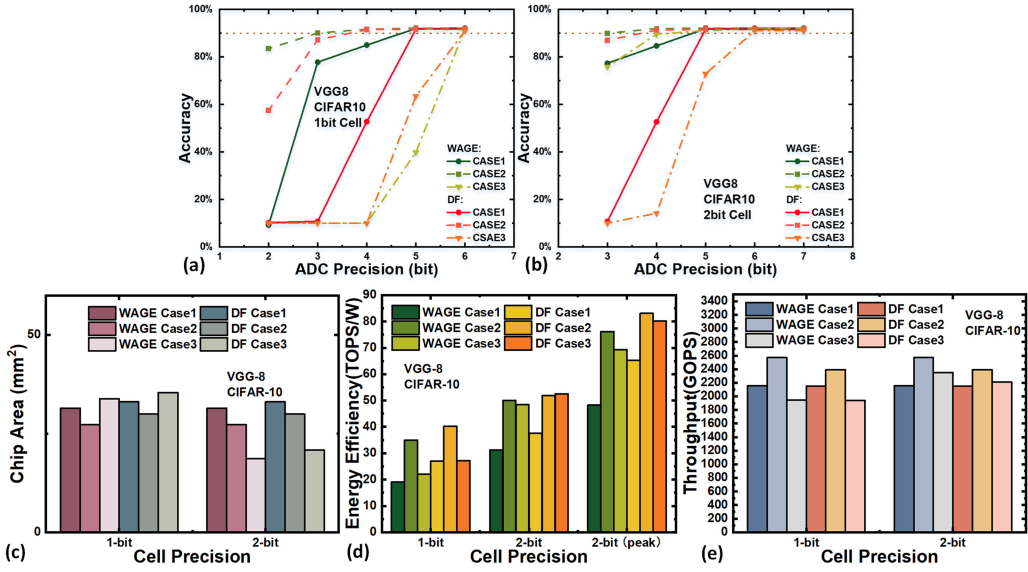


Fig. 12. (a–b) Accuracy performance vs. ADC precision for 2b-W/8b-IN quantized VGG8 (c–e) Hardware performance (energy efficiency, throughput, area overhead) vs. different design options (quantization methods, mapping methods, cell precision) for 2b-W/8b-IN quantized VGG8.

digital processing. In general, according to these results and analysis, we can infer that nonlinear ADC quantization could not effectively benefit CIM designs, which may cause additional problems.

As shown in Figure 12, we further evaluate the ADC quantization and hardware performance of the 2-bit weight and 8-bit input (2b-W/8b-IN) quantized VGG8 network as a more aggressively quantized illustration. Under the same cell precision, Figure 12(a), (b) shows that the ADC quantization trend for 2b-W/8b-IN case is similar to the 8b-W/8b-IN but slightly worse for all mapping strategies. On the contrary, for the hardware evaluation as shown in Figure 12(c–e), we could see the 1-bit cell performance for this 2b-W/8b-IN case is similar to the 4-bit cell case under the 8-bit weight network instead of the 1-bit cell case. The reasons are complicated and many-sided, but the main cause is the following two sides. On one side, the 2-bit quantization gives a bigger difference between the ADC precision of different mapping methods, making Case2 mapping more efficient than the other two in the ADC step. On the other side, there are more “1”s in the weights array for Case1 as -1 is mapped to “11” while the other two cases eliminate the “11” pattern in weights. From this observation, the performance is more likely to follow the relative cell precision to the weight precision instead of the absolute level. When the precision increases to 2-bit for the bit weight, the trend becomes very different for this extreme case. For Case1, increasing the cell precision from 1-bit to 2-bit will not further reduce the number of subarrays needed per weight array. Even when a 2-bit cell is adopted, there will only be weight digits of 0 and 1. However, when the 2bit cells are used, the digit 1 will be mapped to a lower conductance value, thus consuming less power. Similarly, there will be no change in subarray numbers for Case2, but 2-bit cells will consume less power because of the lower conductance mapping. The only option that will cause a big hardware resource difference is the Case3 since the number of subarrays needed is exactly $\lceil N/k \rceil$. In the 2-bit cell case, only one CIM array is needed for one weight array, making the mapping very efficient. From the array-only result, the Case2 is still better (as shown by the 2bit(peak) result in Figure 12(d)). On the other hand, the power consumption of digital peripherals, especially the interconnect, becomes comparable with the CIM for the extreme case. Thus, Case3 has slightly

better energy efficiency because of smaller interconnecting power from the smaller area for the DF quantized network.

5 CONCLUSION

In this work, we explored the design space of the CIM in terms of algorithm-to-hardware mapping. We demonstrated the basic design flow of neural network mapping and explored two software quantization methods (i.e., DF and WAGE) combined with different MAC mapping approaches (cell precision, number representation methods). By analyzing the hardware performance, we find that the quantization method with high input sparsity will help to improve the energy efficiency of the CIM accelerators, which is the DF according to our assessment. Compared to the input distribution, the weight distribution is less affected by different quantization methods utilized in this work, thus leading to similar energy consumption. However, increasing the weight sparsity could also improve the energy efficiency, thus other techniques such as pruning could further be introduced. In the extreme case of weight precision vs. cell precision, the mapping strategy may also affect the digits' sparsity and thus the energy consumption. In conclusion, sparsity could improve the energy efficiency of the CIM architecture without any special techniques such as zero skipping. When choosing the networks mapped to the CIM accelerator, improving the sparsity of both input and weight digits is desired for better hardware performance.

We also evaluated the hardware performance with and without ADC quantization loss across different design options. Low precision ADC could improve the hardware performance. Thus, it is desired to have as much ADC precision loss as possible while maintaining the software performance. When mapping the same network to different architectures, its ADC requirements could differ across the mapping schemes utilized. The differential pair scheme could tolerate the most ADC quantization loss across all cell precisions while 2's complement and shifted unsigned INT require high precision for high precision cells and low precision cells separately. After further analysis, we conclude that a mapping scheme with identical distribution among weight/input digits and components will be a better choice considering ADC quantization. We also found that while the nonlinear ADC quantization could achieve 1-bit more ADC quantization loss for some tasks, this trend is not guaranteed for all the tasks because of the initial accuracy loss. The nonlinear ADC quantization is not recommended for the CIM accelerators considering the hardware cost and difficulties in implementing nonlinear references.

For CIM designs with high-precision weights mapped to low-precision cells, 2's complement number representation is the best choice, regardless of whether ADC quantization loss is introduced. Considering the CIM designs equipped with memory cells that have comparable precisions with weight precisions, the shifted unsigned INT is the best choice with full precision ADC, while the differential pair should be selected if ADC loss is introduced. Under the extreme case that weight precision equals or is even lower than the cell precisions, the differential pair and shifted unsigned INT could be comparable because of the peripheral digital circuits. For the analysis purpose, when designing the architecture for CIM, the main components that will affect the hardware performance are cells needed per weight synapse (or subarray needed per weight array, the input/weight digital sparsity, ADC precision, and the interconnects. The higher the input/weight digit sparsity and the cells need per weight synapse, and the lower the ADC precision, the better the result will be. As the CIM-array becomes more and more energy-efficient, the interconnect will show up and become the accelerator's bottleneck. The mapping strategy decides the cells needed per weight synapse under certain cell precision. The digit sparsity/distribution could be affected by the network parameter statistics and the also the mapping strategy. The input/weight digit distribution mainly decides the ADC. Finally, the interconnect is mainly decided by the chip size considering the routing length, which could be highly decided by the subarray needed per weight array.

Based on our results on evaluating the CIM system's performance with various design options, optimized design space for neural network mapping could improve energy efficiency by $\sim 2\times$ and throughput by $1.2\sim 1.6\times$ while reducing $5\%\sim 25\%$ area overhead. Our work is the most comprehensive study so far documenting algorithm-hardware mapping strategies in CIM accelerator design. Our findings and demonstrations have provided a deeper insight into the design space exploration of system-level CIM design. When switching to other architectures, the focus should be on selecting the mapping strategies and analyzing their impact on hardware requirements, as we evaluated in this work. While extending our design space exploration strategy to new applications or quantization methods, we should examine the weight and input distribution change since they play a significant role in hardware performance. Thus, the same strategy could be applied to different architectures or applications to improve the system performance accordingly. Future research should thoroughly examine the mutual effect between system-level design options and circuit/device techniques. It would be of great help in exploring how non-idealities, such as conductance variation [36], or limitations from circuit design, such as the ADC offset [37], affect the selections of system design options.

REFERENCE

- [1] S. Yu. 2018. Neuro-inspired computing with emerging nonvolatile memorys. *Proceedings of the IEEE* 106, 2 (2018), 260–285.
- [2] W. Wu, H. Wu, B. Gao, P. Yao, X. Zhang, X. Peng, S. Yu, and H. Qian. 2018. A methodology to improve linearity of analog RRAM for neuromorphic computing. In *IEEE Symposium on VLSI Technology (VLSI'18)*. Honolulu, HI, USA, 2018.
- [3] M. Jerry, P.-Y. Chen, J. Zhang, P. Sharma, K. Ni, S. Yu, and S. Datta. 2017. Ferroelectric FET analog synapse for acceleration of deep neural network training. In *IEEE International Electron Devices Meeting (IEDM'17)*. San Francisco, CA, USA, 2017.
- [4] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramonian, John Paul Strachan, Miao Hu, R. Stanley Williams, and Vivek Srikumar. 2016. ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars. In *ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. Seoul, Korea (South) 2016.
- [5] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. 2016. PRIME: A novel processing-in-memory architecture for neural network. In *International Symposium on Computer Architecture*. Seoul, Korea (South), 2016.
- [6] Shibin Tang, Shouyi Yin, Shixuan Zheng, Peng Ouyang, Fengbin Tu, Leiyue Yao, JinZhou Wu, Wenming Cheng, Leibo Liu, and Shaojun Wei. 2017. AEPE: An area and power efficient RRAM crossbar-based accelerator for deep CNNs. In *IEEE 6th Non-Volatile Memory Systems and Applications Symposium (NVMSA)*. Hsinchu, Taiwan, 2017.
- [7] Jacob Benoit, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Conference on Computer Vision and Pattern Recognition*. Salt Lake City, UT, USA, 2018.
- [8] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. MobileNets: Efficient convolutional neural networks for mobile vision applications. In *arXiv*, 2017.
- [9] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun. 2018. ShuffleNet: An extremely efficient convolutional neural network for mobile devices. In *Conference on Computer Vision and Pattern Recognition*. Salt Lake City, UT, USA, 2018.
- [10] M. Courbariaux, Y. Bengio, and J.-P. David. 2015. Training deep neural networks with low precision multiplications. In *Workshop Contribution at International Conference on Learning Representations (ICLR)*. San Diego, CA, USA, 2015.
- [11] Ron Banner, Itay Hubara, Elad Hoffer, and Daniel Soudry. 2018. Scalable methods for 8-bit training of neural networks. In *Neural Information Processing Systems*. Montreal, Quebec, Canada, 2018.
- [12] Shuang Wu, Guoqi Li, Feng Chen, and Luping Shi. 2018. Training and inference with integers in deep neural networks. In *International Conference on Learning Representations (ICLR)*. Vancouver, British Columbia, Canada, 2018.
- [13] I. Hubara, M. Courbariaux, D Soudry, R. El-Yaniv, and Y. Bengio. 2016. Binarized neural networks. In *30th Conference on Neural Information Processing Systems (NIPS)*. Barcelona, Spain, 2016.

- [14] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi. 2016. XNOR-Net: ImageNet classification using binary convolutional neural networks. In *14th European Conference on Computer Vision (ECCV)*. Amsterdam, the Netherlands, 2016.
- [15] Yukuan Yang, Lei Deng, Shuang Wu, Tianyi Yan, Yuan Xie, and Guoqi Li. 2020. Training high-performance and large-scale deep neural networks with full 8-bit integers. *Neural Networks* 125 (2020), 70–82.
- [16] S. Esser, J. McKinstry, D. Bablani, R. Appuswamy, and D. Modha. 2020. Learned step size quantization. In *International Conference on Learning Representations (ICLR)* 2020.
- [17] Sangil Jung, Changyong Son, Seohyung Lee, Jinwoo Son, Youngjun Kwak, Jae-Joon Han, Sung Ju Hwang, and Changkyu Choi. 2019. Learning to quantize deep networks by optimizing quantization intervals with task loss. In *IEEE Conference on Computer Vision and Pattern Recognition*. Long Beach, CA, USA, 2019.
- [18] Songyun Qu, Bing Li, Ying Wang, Dawen Xu, Xiandong Zhao, and Lei Zhang. 2020. RaQu: An automatic high-utilization CNN quantization and mapping framework for general-purpose RRAM accelerator. In *ACM/IEEE Design Automation Conference (DAC)*. San Francisco, CA, USA, 2020.
- [19] Zhenhua Zhu, Hanbo Sun, Yujun Lin, Guohao Dai, Lixue Xia, Song Han, Yu Wang, and Huazhong Yang. 2019. A configurable multi-precision CNN computing framework based on single bit RRAM. In *ACM/IEEE Design Automation Conference (DAC)*. Las Vegas, NV, USA, 2019.
- [20] Zhenhua Zhu, Jilan Lin, Ming Cheng, Lixue Xia, Hanbo Sun, Xiaoming Chen, Yu Wang, and Huazhong Yang. 2018. Mixed size crossbar based RRAM CNN accelerator with overlapped mapping method. In *International Conference on Computer-Aided Design (ICCAD)*. San Diego, CA, USA, 2018.
- [21] Hongwu Jiang, Shanshi Huang, Xiaochen Peng, Jian-Wei Su, Yen-Chi Chou, Wei-Hsing Huang, et al. 2020. A two-way SRAM array based accelerator for deep neural network on-chip training. In *ACM/IEEE Design Automation Conference (DAC)*. San Francisco, CA, USA, 2020.
- [22] Jian-Wei Su, Xin Si, Yen-Chi Chou, Ting-Wei Chang, Wei-Hsing Huang, et al. 2020. A 28nm 64Kb inference-training two-way transpose multibit 6T SRAM compute-in-memory macro for AI edge chips. In *International Solid- State Circuits Conference*. San Francisco, CA, USA, 2020.
- [23] Xin Si, Jia-Jing Chen, Yung-Ning Tu, Wei-Hsing Huang, Jing-Hong Wang, Yen-Cheng Chiu, et al. 2019. A twin-8T SRAM computation-in-memory macro for multiple-bit CNN-based machine learning. In *International Solid- State Circuits Conference*. San Francisco, CA, USA, 2019.
- [24] Cheng Ming, Xia Lixue, Zhu Zhenhua, Cai Yi, Xie Yuan, Wang Yu, and Yang Huazhong. 2019. TIME: A training-in-memory architecture for RRAM-based deep neural networks. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 38, 5 (2019), 834–847.
- [25] Weier Wan, Rajkumar Kubendran, Clemens Schaefer, Sukru Burc Eryilmaz, Wenqiang Zhang, Dabin Wu, Stephen Deiss, Priyanka Raina, He Qian, Bin Gao, Siddharth Joshi, Huaqiang Wu, H.-S. Philip Wong, and Gert Cauwenberghs. 2022. A compute-in-memory chip based on resistive random-access memory. *Nature* (2022), 504–512.
- [26] Hongwu Jiang, Wantong Li, Shanshi Huang, and Shimeng Yu. 2022. A 40nm analog-input ADC-free compute-in-memory RRAM macro with pulse-width modulation between sub-arrays. In *IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)*. Honolulu, HI, USA, 2022.
- [27] Song Han, Huizi Mao, and William J. Dally. 2016. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. In *International Conference on Learning Representations*. San Juan, Puerto Rico, 2016.
- [28] Songyun Qu, Bing Li, Ying Wang, and Lei Zhang. 2021. ASBP: Automatic structured bit-pruning for RRAM-based NN accelerator. In *ACM/IEEE Design Automation Conference (DAC)*. San Francisco, CA, USA, 2021.
- [29] Hanbo Sun, Zhenhua Zhu, Yi Ca, Xiaoming Chen, Yu Wang, and Huazhong Yang. 2020. An energy-efficient quantized and regularized training framework for processing-in-memory accelerators. In *Asia and South Pacific Design Automation Conference (ASP-DAC)*. Beijing, China, 2020.
- [30] Xiaochen Peng, Shanshi Huang, Hongwu Jiang, Anni Lu, and Shimeng Yu. 2021. DNN+NeuroSim V2.0: An end-to-end benchmarking framework for compute-in-memory accelerators for on-chip training. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 40, 11 (2021), 2306–2319.
- [31] Wantong Li, Shanshi Huang, Xiaoyu Sun, Hongwu Jiang, and Shimeng Yu. 2021. Secure-RRAM: A 40nm 16kb compute-in-memory macro with reconfigurability, sparsity control, and embedded security. In *Custom Integrated Circuits Conference*. Austin, TX, USA, 2021.
- [32] Xiaoyu Sun, Shihui Yin, Xiaochen Peng, Rui Liu, Jae-sun Seo, and Shimeng Yu. 2018. XNOR-RRAM: A scalable and parallel resistive synaptic architecture for binary neural networks. In *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. Dresden, Germany, 2018.
- [33] Anni Lu, Xiaochen Peng, Wantong Li, Hongwu Jiang, and Shimeng Yu. 2021. NeuroSim simulator for compute-in-memory hardware accelerator: Validation and benchmark. *Frontiers in Artificial Intelligence* 2021.

- [34] Shihui Yin, Yulhwa Kim, Xu Han, Hugh Barnaby, Shimeng Yu, Yandong Luo, Wangxin He, Xiaoyu Sun, Jae-Joon Kim, and Jae-sun Seo. 2019. Monolithically integrated RRAM- and CMOS-based in-memory computing optimizations for efficient deep learning. *IEEE Micro* 39, 6 (2019), 54–63.
- [35] Jinshan Yue, Xiaoyu Feng, Yifan He, Yuxuan Huang, Yipeng Wang, Zhe Yuan, Mingtao Zhan, Jiaxin Liu, Jian-Wei Su, Yen-Lin Chung, Ping-Chun Wu, Li-Yang Hung, Meng-Fan Chang, Nan Sun, Xueqing Li, Huazhong Yang, et al. 2021. A 2.75-to-75.9TOPS/W computing-in-memory NN processor supporting set-associate block-wise zero skipping and ping-pong CIM with simultaneous computation and weight updating. In *IEEE International Solid-State Circuits Conference (ISSCC)*. San Francisco, CA, USA, 2021.
- [36] Marco Donato, Brandon Reagen, Lillian Pentecost, Udit Gupta, David Brooks, and Gu-Yeon Wei. 2018. On-chip deep neural network storage with multi-level eNVM. In *ACM/ESDA/IEEE Design Automation Conference (DAC)*. San Francisco, CA, USA, 2018.
- [37] Shanshi Huang, Xiaochen Peng, Hongwu Jiang, Yandong Luo, and Shimeng Yu. 2021. Exploiting process variations to protect machine learning inference engine from chip cloning. In *IEEE International Symposium on Circuits and Systems (ISCAS)*. Daegu, Korea (South), 2021.

Received 20 March 2022; revised 10 October 2022; accepted 23 October 2022